

Claude Code Handoff — GCN Cargo Container Token & Traceability

TESTNET / DEVNET ONLY · PRE-AUDIT · PROOF-OF-CONCEPT · NOT DEPLOYED FOR LIVE VALUE	
Module	Cargo Container Tokenization + Traceability Ledger
Platform	GCN Exchange — addition to the existing exchange platform
Chains	Hedera testnet · Solana devnet · EVM testnet (Sepolia, Hedera JSON-RPC relay 296)
Status	Proof-of-concept. Pre-audit. Testnet only. Not deployed for live value.
Date	2 October 2026
Author	Claude Code session, for James A. Holmes Jr.

1. What this module does

It gives a shipping container a token identity keyed to its ISO 6346 id, and an append-only ledger of where that container has been attested to be — with distance accounting and an explicit record of how each position was obtained.

It answers three questions:

Question	Where it is answered
Which container is this?	ISO 6346 id + check digit, one token per id
Where has it been?	<code>ContainerJourney</code> waypoint log / HCS topic, port calls in order
How far has it travelled?	<code>cumulativeMeters</code> , reported in miles / nautical miles / km

2. The honesty constraint this was built around

The brief that produced this module was: *"A token does not prove a physical container exists. Please create this ability."*

That ability cannot be fully created, and the design says so rather than pretending otherwise. A blockchain cannot observe a steel box. No contract, key or consensus mechanism changes that. Anyone who tells you their container token proves physical existence is either confused or selling something.

What *can* be built — and what is built here — is a record whose weaknesses are legible instead of hidden. The mechanism is the **evidence tier**: every waypoint carries a machine-readable statement of what was actually observed.

Tier	What it observed	Trust weight
<code>TIER1_CONTAINER_TELEMETRY</code>	the container itself — GPS/BLE device on the box	1.00
<code>TIER2_CARRIER EDI</code>	a handling event in a carrier/terminal system of record (CODECO, BAPLIE, B/L)	0.70
<code>TIER3_VESSEL AIS</code>	the ship , not the box	0.40
<code>TIER4_MANUAL</code>	nothing — a person asserted it	0.15

Only TIER1 observes the container. This matters more than it sounds: almost every "container tracking" claim in the market is TIER3 — a vessel's AIS transponder, with the container's position inferred from the ship's. That inference is only valid while the container is demonstrably aboard that ship. Outside a `LOADED_VESSEL ... DISCHARGED` window it is a ship's position wearing a container's name, and both engines flag it as `ORPHAN_VESSEL_POSITION`.

`hasContainerLevelEvidence(tokenId)` returns `false` when nothing in a journey observed the container. An interface built on this should show that prominently, not bury it.

Distance is a lower bound, always

`cumulativeMeters` is the sum of **great-circle hops between attested positions**. It understates reality for two reasons:

- Ships do not sail great circles — they follow traffic lanes, canals, coastlines and weather routing. Shanghai→Rotterdam is ~10,500 nm by arc and ~11,800 nm via Suez.
- The log is sparse. Positions between attestations are unknown, and a sparse log cuts corners off the real track.

So it is reported as **"attested great-circle distance"**, never "distance travelled". `ContainerJourney.distanceCaveat()` keeps that sentence on-chain next to the number, so no interface can quote the figure and claim the caveat came from somewhere else.

3. What is verified, and what is not

This is the section to read before quoting any of this to a counterparty.

Tested and passing (run in this session)

Check	Result
EVM contract suite — <code>ContainerToken</code> + <code>ContainerJourney</code>	18 tests passing
Geodesy and journey replay — <code>shared/test-geo.js</code>	14 checks passing
Agent suite — screening + journey audit	15 tests passing
Solidity compile, solc 0.8.19, optimizer on	clean, no warnings (<code>ContainerToken</code> 11,505 B · <code>ContainerJourney</code> 10,164 B)
JS ↔ Python haversine agreement	agree to under 1 metre on three known pairs
Skill structure validator	passes

The EVM distance test uses real port coordinates and real haversine values, and asserts the Shanghai→Singapore→Jebel Ali total lands between 9,000 and 13,000 km — a sanity check against the real world, not against my own arithmetic.

Compiles and parses but has never run

- **Hedera scripts** (`create-collection` , `mint-container` , `attest` , `log-waypoint` , `journey`) — syntax-checked only. Never executed against Hedera testnet. Needs a portal account.
- **Solana scripts** (`mint-container` , `log-waypoint`) — syntax-checked only. Never executed against devnet.
- `deploy.js` / `deploy-journey.js` — parse; never run against a live testnet RPC.
- **The agents with a live model** — the deterministic paths are tested; neither agent has been run against Claude, so the prompt/output quality is unproven.
- `hedera/src/journey.js` **mirror-node read** — the sandbox blocks the mirror node host, so the HTTP path is untested. The replay logic it feeds is tested.

Sample data, not real

`shared/sample-container.json` and `agent/demo_route.py` use the standard ISO 6346 worked example (`CSQU3054383`), an illustrative IMO, an illustrative MTI of 4, and approximate port centroids. **None of it is verified by GCN.** `shared/unlocode.js` is hand-typed demo data and must be replaced with the official UNECE UN/LOCODE list.

A finding from verification worth recording

The JS replay and the Python auditor were written to the same doctrine but drifted: the Python side caught a vessel-AIS position recorded after discharge and the JS side did not. The demo route has that defect planted in it, which is how it surfaced. `geo.js` now carries the `ORPHAN_VESSEL_POSITION` check and a test for it. **Any future change to one engine needs the same change in the other** — the cross-engine haversine test exists to make that drift fail loudly.

4. File map

```
cargo-container-token/
├── shared/
│   ├── iso6346.js           check-digit validation (TESTED)
│   ├── geo.js              haversine, bearings, journey replay, anomalies (TESTED)
│   ├── unlocode.js         demo port table – REPLACE with the official list
│   ├── test-geo.js         14 offline checks
│   ├── demo-journey.js     full sample report, no keys needed
│   └── sample-container.json SAMPLE DATA
├── contracts/evm/
│   ├── contracts/ContainerToken.sol   ERC-721 identity + MTI-gated transfer
│   ├── contracts/ContainerJourney.sol  append-only traceability ledger
│   ├── test/*.test.js                18 tests (TESTED)
│   ├── scripts/deploy.js             refuses mainnet names
│   ├── scripts/deploy-journey.js     --seed writes a sample route
│   └── hardhat.config.js             SOLC_JS_PATH offline-compile override
├── hedera/src/
│   ├── client.js                   testnet only, no mainnet path exists
│   ├── create-collection.js        HTS NFT collection + HCS topic
│   ├── mint-container.js           metadata "ISO|hash16" (100-byte HTS cap)
│   ├── attest.js                  vessel MTI attestation → HCS
│   ├── log-waypoint.js            one waypoint → HCS, hash-chained
│   └── journey.js                 mirror-node read + full report (verification path)
├── solana/src/
│   ├── mint-container.js          0-decimal supply-1 SPL, mint authority revoked
│   └── log-waypoint.js            Memo waypoint – NO on-chain validation
├── agent/
│   ├── container_agent.py         MTI vessel screening (Atomic Agents)
│   ├── journey_agent.py          journey audit + confidence grading
│   ├── demo_route.py             SAMPLE route with two planted defects
│   └── test/*.py                 15 offline tests
└── skill/gcn-container-tokenization/  packaged skill, validator-clean
```

5. What `ContainerJourney.sol` actually enforces

Worth knowing precisely, because this is what distinguishes it from a database:

- **Append-only.** No edit function. No delete function. `appendCorrection` records that an earlier waypoint is wrong and leaves the original exactly where it is. A ledger whose history can be tidied up is not evidence of anything.
- **Chronological order** — each waypoint must postdate the last.
- **No future timestamps** (1-hour clock-skew grace).

- **Evidence hash required** — a waypoint with no source document is rejected outright.
- **Per-mode speed ceiling** — SEA 15.5 m/s (~30 kn), ROAD 35, RAIL 45, AIR 300, YARD 3. A leg no conveyance could cover in the elapsed time reverts. Per-mode because containers are multimodal: a 95 km leg in 30 minutes is impossible by sea and ordinary by rail, and the contract gets that right.
- **A geometric band on the claimed distance, without trig.** One degree of latitude is ~111,320 m everywhere; one of longitude is that at most. So `|dLat|` is a floor and `|dLat|+|dLon|` a ceiling on the true haversine value, with 1% slack. Catches a transposed sign or a mistyped coordinate on-chain. It is a sanity band, **not** a substitute for recomputing haversine off-chain.
- **Container-level observations counted separately**, so "did anything observe the box" is one call.
- **Token must exist** — the append reverts if `ownerOf(tokenId)` does.

Deliberately *not* enforced: haversine itself (no cheap on-chain trig), and anything about whether the observer is telling the truth. The contract constrains arithmetic and order. It cannot constrain honesty, and the tier system is the honest accounting of that.

6. How to run it

Offline — no keys, no network, works anywhere

```
node shared/test-geo.js          # 14 geodesy / replay checks
node shared/demo-journey.js      # sample report, planted defects visible
cd agent && pip install atomic-agents anthropic instructor pytest && python -m pytest -q
```

EVM tests

```
cd contracts/evm
npm install
npx hardhat test
```

If `binaries.soliditylang.org` is blocked (locked-down CI, sandboxes), compile from a local solc-js build instead — this is how the suite was run in this session:

```
npm i solc@0.8.19
SOLC_JS_PATH=$(pwd)/node_modules/solc/soljson.js npx hardhat test
```

EVM testnet deploy

```
export TESTNET_PRIVATE_KEY=0x...          # testnet key ONLY
npx hardhat run scripts/deploy.js --network sepolia
export CONTAINER_TOKEN_ADDRESS=0x...
npx hardhat run scripts/deploy-journey.js --network sepolia -- --seed
```

`deploy.js` refuses any network whose name contains mainnet, polygon, base, arbitrum, optimism, bsc or avalanche.

Hedera testnet

```
cd hedera && npm install && cp .env.example .env # fill from portal.hedera.com
node src/create-collection.js                  # writes state.json
node src/mint-container.js CSQU3054383
node src/log-waypoint.js --iso CSQU3054383 --event LOADED_VESSEL \
  --tier TIER2_CARRIER EDI --port CNSHA --at 2026-03-02T14:00:00Z --evidence ./codeco.edi
node src/journey.js CSQU3054383                # verification report
```

`journey.js` reads from the public mirror node, so anyone can run it against a topic id with no credentials and get the same answer. That reproducibility is the only sense in which the record is worth anything.

Solana devnet

```
cd solana && npm install
node src/mint-container.js CSQU3054383
node src/log-waypoint.js --iso CSQU3054383 --event LOADED_VESSEL \
  --tier TIER2_CARRIER EDI --port CNSHA --evidence ./doc.pdf
```

Environment variables

Variable	Used by	Note
TESTNET_PRIVATE_KEY	Hardhat	testnet only — never a mainnet key
SEPOLIA_RPC_URL	Hardhat	defaults to <code>rpc.sepolia.org</code>
Hedera_TESTNET_RPC	Hardhat	defaults to <code>testnet.hashio.io/api</code> (chainId 296)
Hedera_TESTNET_ACCOUNT_ID , Hedera_TESTNET_PRIVATE_KEY	Hedera SDK	from portal.hedera.com
Hedera_MIRROR_URL	<code>journey.js</code>	defaults to testnet mirror node
SOLANA_DEVNET_KEYPAIR	Solana	path to a devnet keypair JSON
CONTAINER_TOKEN_ADDRESS	<code>deploy-journey.js</code>	deployed <code>ContainerToken</code>
ANTHROPIC_API_KEY	agents	required for live runs; offline paths do not need it
SOLC_JS_PATH	Hardhat	optional offline-compile override

7. Next steps, in priority order

- Run it on the three testnets.** Nothing on the Hedera or Solana side has executed. This is the gap between "compiles" and "works", and it is the first thing to close.
- Wire one real data source.** Pick carrier EDI (TIER2) first — it is the best trust-per-unit-effort and needs no hardware. An ingester with its own credentials and its own observer key, writing waypoints automatically, is what turns this from a schema into a system. Until then every waypoint is as good as the person running the script.
- Get real Pole Star MTI data.** The MTI is a commercial product; a real score needs a Pole Star account and API access. Every score in this codebase is illustrative. Until this is wired, the screening agent correctly refuses to grade a vessel and returns `needs_mti`.
- Add TIER1 telemetry for at least one lane.** This is the only tier that observes the container, and the one that makes the record meaningfully different from what a freight forwarder already has. One instrumented lane demonstrates more than a hundred AIS-derived ones.
- Split observer custody.** OBSERVER_ROLE is one admin key today. Per-carrier keys are what make an observer identity mean anything.
- Write the Anchor program** so the Solana path has the same invariants as the EVM one. Right now the two chains enforce very different things under the same name.
- Replace the port table** with the official UNECE UN/LOCODE list, recording the edition date.
- Audit before any mainnet consideration.** Non-negotiable. The MTI-gated transfer policy in particular is a lock on an asset and deserves adversarial review.
- Metaplex Token Metadata** on Solana, so the token displays in wallets.
- Decide the legal question:** does the token represent the *record* of a container or *title* to it? The code currently implements the record. Title is a legal construction, not a contract change, and conflating the two is how tokenization projects get into trouble.

8. Open questions for James

- Which chain is canonical?** Three ledgers with different enforcement will disagree eventually. The EVM contract is the only one with on-chain validation — is it the source of truth, with Hedera as the cheap audit log and Solana as notarisation? That reading is coherent; three equal peers is not.
- Who are the observers?** The tier system is only as good as the identities behind the keys. Which carriers, terminals or inspectors would actually hold OBSERVER_ROLE?
- Is there a lane with real telemetry available?** One instrumented route would move this from proof-of-concept to demonstrable.
- Does GCN have or want a Pole Star relationship?** The MTI integration is designed against the public product page. Nothing here implies a partnership, and nothing should until one exists in writing.

9. Constraints carried through the build

- Testnet/devnet only; mainnet paths deliberately absent. The Hedera client has no mainnet branch at all, and the EVM deploy script rejects mainnet network names.
- No invented MTI scores. No implied Pole Star partnership, endorsement or data-sharing agreement.
- No claim that a token proves physical existence, anywhere in the code, comments or output.
- Every distance figure carries its lower-bound caveat — including on-chain.
- Sample data labelled as sample data at the point of use.
- This module is not blended with any other GCN venture material.

Source for all MTI statements: Pole Star Maritime Transparency Index — <https://www.polestarglobal.com/maritime-transparency-index/> (vessel score 0–5; dimensions Vessel, Voyage, Emissions*, Border Security*, Supply Chain* (*in development); 200+ data sources). The MTI scores a **vessel**, not a container or a cargo, which is why it is modelled here as a vessel attestation linked to a container token rather than as a property of the container.